



APPUNTI SINTETICI DI ARCHITETTURA SOFTWARE



Architetture e Modelli Fondamentali

- **Monolite (Monolith):** Un'unica, grande applicazione eseguita su un singolo server. È semplice da avviare e deployare inizialmente, ma soffre di una **difficile scalabilità** (si scala tutto o niente) e rende i team strettamente **accoppiati**. Il deploy è rischioso.
 - **Three-Tier Architecture (Architettura a Tre Livelli):** Separa l'applicazione in tre strati logici: **Presentation Layer** (interfaccia utente), **Business Logic Layer** (logica applicativa) e **Data Layer** (persistenza/database). Offre una buona separazione delle responsabilità.
 - **Microservizi (Microservices):** Architettura basata su tanti servizi **piccoli e indipendenti** che comunicano tra loro. Sono autonomi nello sviluppo, nel deploy e nella scalabilità. Offrono **fault isolation** (l'errore di un servizio non blocca gli altri), ma aumentano la **complessità di rete** e rendono difficile la **data consistency**.
 - **Event-Driven Architecture (EDA):** Architettura guidata da **eventi** (cambiamenti di stato significativi). La comunicazione è prevalentemente **asincrona**. Questo garantisce un **alto disaccoppiamento** e una grande reattività, ma rende il debugging e la tracciabilità delle operazioni più complessi (**Eventual Consistency**).
-



Scalabilità e Performance

- **Scalabilità:** La capacità di un sistema di gestire un carico di lavoro crescente. Si distingue in:
 - **Scalabilità Verticale (Scaling Up):** Aggiungere potenza a un singolo server (più RAM, CPU). È semplice ma ha un **limite fisico** e crea un **Single Point of Failure (SPOF)**.
 - **Scalabilità Orizzontale (Scaling Out):** Aggiungere più server (istanze) per distribuire il carico. Non ha limiti teorici e offre **alta disponibilità (HA)**, ma è architetturealmente più complessa.
 - **Load Balancer:** Componente cruciale che distribuisce le richieste in entrata tra le diverse istanze backend, garantendo l'HA e prevenendo i sovraccarichi.
 - **CDN (Content Delivery Network):** Una rete di server distribuiti geograficamente che memorizza e serve **contenuti statici** (immagini, CSS, JS) all'utente dalla posizione più vicina, riducendo la **latenza**.
 - **Resilienza:** La capacità del sistema di **riprendersi dagli errori** e continuare a funzionare, essenziale per la Fault Tolerance.
-

Autenticazione e Stato

- **Stateless vs. Stateful:** I sistemi **Stateless** (senza stato, come HTTP) non mantengono memoria delle interazioni passate, ogni richiesta è indipendente. Sono ideali per la scalabilità orizzontale. I sistemi **Stateful** mantengono una memoria (es. sessioni sul server), richiedendo session storage centralizzato.
 - **JWT Token (JSON Web Token):** Token crittograficamente **autofirmato** che contiene le informazioni necessarie per identificare l'utente (**claims**). Permette un'autenticazione **stateless** altamente scalabile, eliminando la necessità di un database di sessioni.
 - **OAuth:** Un protocollo di **autorizzazione delegata** che consente a un'applicazione di accedere a risorse su un altro servizio (es. Google, Facebook) per conto dell'utente, senza condividere la sua password.
 - **SSO (Single Sign-On):** Permette all'utente di accedere a più applicazioni o servizi con un **unico set di credenziali**.
-

Database e Consistenza

- **Teorema CAP:** Afferma che in un sistema distribuito si possono garantire solo **due delle tre** proprietà contemporaneamente: **Consistency** (Consistenza immediata), **Availability** (Disponibilità) e **Partition Tolerance** (Tolleranza alle partizioni di rete).
 - **ACID:** Le proprietà fondamentali delle transazioni nei database relazionali: **Atomicità** (tutto o niente), **Coerenza** (stato valido), **Isolamento** (non interferenza), **Durabilità** (permanenza). Prioritizzano la consistenza.
 - **BASE:** Modello alternativo per sistemi distribuiti: **Basically Available** (disponibilità di base), **Soft State** (stato variabile), **Eventual Consistency** (consistenza nel tempo). Prioritizzano disponibilità e tolleranza alle partizioni.
 - **Eventual Consistency:** I dati su tutti i nodi **convergeranno** allo stesso valore dopo un certo periodo, a meno che non ci siano nuovi aggiornamenti. Accettata in molti contesti non critici.
 - **Database per Servizio:** Il pattern dove ogni microservizio possiede il proprio database **privato**, garantendo autonomia e la libertà di usare tecnologie diverse.
-

Comunicazione tra Servizi

- **Comunicazione Sincrona (es. REST):** Il chiamante **attende la risposta**. È semplice e immediata, ma crea **accoppiamento temporale** e può portare a *cascading failures* (fallimenti a catena).
- **Comunicazione Asincrona (es. Message Queues):** Il chiamante **non attende** e continua l'esecuzione. Offre **disaccoppiamento totale** e resilienza, ma introduce complessità.
- **REST APIs:** Architettura basata su HTTP, orientata alle risorse, usa i metodi standard (GET, POST, PUT, DELETE). Semplice e **cache-able**.

- **GraphQL:** Un linguaggio di query che permette al client di richiedere **esattamente i dati necessari**, risolvendo il problema dell'*over-fetching* tipico di REST.
 - **Message Queues / Pub/Sub:** Meccanismi asincroni che garantiscono **decoupling** (separazione tra servizi) e **affidabilità** (i messaggi vengono salvati e non si perdono), fondamentali per la scalabilità in microservizi.
-

Resilienza e Fault Tolerance

- **Circuit Breaker (Interruttore):** Un pattern che blocca le chiamate a un servizio che sta **fallendo ripetutamente**. Ha tre stati (Closed, Open, Half-Open) e serve a prevenire i *cascading failures* e a dare tempo al servizio di recuperare.
 - **Retry Pattern:** Strategia per **riprovare** richieste fallite, spesso con un **Exponential Backoff** (ritardo crescente) per non sovraccaricare il servizio in ripresa.
 - **Bulkhead Pattern:** Isola le risorse per evitare che il fallimento di un componente (ad esempio, un pool di thread) comprometta l'intero sistema (come i compartimenti stagni di una nave).
 - **Service Discovery:** Il meccanismo che consente ai servizi di **trovare automaticamente** gli indirizzi degli altri servizi in una rete dinamica (essenziale in Kubernetes e microservizi).
-

Container, Orchestrazione e Complessità

- **Container (Docker):** Unità software che impacchetta codice e **tutte le sue dipendenze** in modo che l'applicazione venga eseguita in modo **consistente** su qualsiasi ambiente. Offre **portabilità** e **isolamento**.
 - **Kubernetes (K8s):** Un sistema di **orchestrazione** dei container. Gestisce l'**auto-scaling**, il **deployment automatico**, il **load balancing** e il **self-healing** (riavvio automatico dei container falliti).
 - **Complessità Essenziale vs. Accidentale:** La complessità **Essenziale** è intrinseca al problema di business e non eliminabile. La complessità **Accidentale** è introdotta da scelte tecnologiche o processuali non ottimali e deve essere minimizzata.
 - **Conway's Law:** Afferma che l'architettura del sistema tenderà a **rispecchiare la struttura comunicativa** dell'organizzazione (i confini dei team diventano i confini dei servizi).
-



Observability, Security e Governance

- **Observability - I Tre Pilastri:** La capacità di comprendere lo stato interno di un sistema analizzando i suoi output: **Metrics** (numeri aggregati come latenza), **Logs** (eventi discreti) e **Traces** (il percorso completo di una richiesta attraverso tutti i servizi).
- **SLO (Service Level Objective):** L'obiettivo di performance concordato per un **SLI** (Service Level Indicator, la metrica che misuriamo). Se non rispettato, entra in gioco l'**SLA** (Service Level Agreement), il contratto formale.
- **Chaos Engineering:** La disciplina di **sperimentare guasti in produzione (Fault Injection)** per costruire fiducia nella resilienza del sistema in condizioni turbolente.
- **Zero Trust Architecture:** Un principio di sicurezza che richiede di **non fidarsi mai** di utenti o servizi, né interni né esterni, e di **verificare sempre** l'identità e l'autorizzazione di ogni richiesta.
- **RTO (Recovery Time Objective):** Il **tempo massimo** accettabile per ripristinare un servizio dopo un disastro.
- **RPO (Recovery Point Objective):** La quantità **massima di dati** che l'organizzazione può permettersi di perdere.
- **ADR (Architecture Decision Records):** Documenti che registrano in modo formale e tracciabile le **decisioni architetturali chiave** (con contesto, opzioni e conseguenze).